udp FACULTAD DE
INGENIERÍA Y CIENCIAS

Estructura de Datos: Laboratorio 3

Nombre(s) Estudiante(s)
Profesor: Alvaro Valdebenito
Ayudante: Diego Banda
Sección 4

14 de Junio 2024

Índice

1. Introducción **3**

2. Desarrollo **4**

2.1. Metodos de Ordenamiento 5

2.2. Tiempos de ejecucion 14

2.3. Arbol de Busqueda Binaria 15

3. Análisis **20**

4. Conclusión **21**

1. Introducción

En el siguiente informe, analizaremos y explicaremos la creación de un código para un buscador de música, diseñado en Java. Para esto, realizaremos diversas implementaciones, como las **Estructuras de Datos**: *ArrayList, una Base de Datos y un BST (Árbol Binario de Búsqueda)*. Además, utilizaremos varios **Métodos de Ordenamiento**, que analizaremos uno a uno más adelante. Estos son: *BubbleSort, InsertionSort, MergeSort y QuickSort*. Finalmente, para analizar su comportamiento, implementaremos la lectura de un archivo CSV, el cual contiene una cantidad significativa de canciones.

2. Desarrollo

A continuación explicaremos uno a uno los métodos del código con el cual se realiza la búsqueda dentro del reproductor.
Primero, implementamos la clase **Cancion**, la cual representa una canción con los atributos: *id, nombre, artista y duración* (en milisegundos).

```java
class Cancion {
    public int id;
    public String nombre;
    public String artista;
    public double duracion; // en milisegundos

    public Cancion(int id, String nombre, String artista, double duracion) {
        this.id = id;
        this.nombre = nombre;
        this.artista = artista;
        this.duracion = duracion;
    }
}
```

Figura 1: Ejemplo de inserción de imagen

Luego es creada la **Base de Datos** en la cual serán implementados cada uno de los métodos del código.

```java
class BaseDeDatos {

    private List<Cancion> listaDeCanciones;
    private NodoArbol rootPorNombre;
    private NodoArbol rootPorDuracion;

    public BaseDeDatos() {
        listaDeCanciones = new ArrayList<>();
    }

    public void agregarCancion(Cancion song) {
        listaDeCanciones.add(song);
    }

    public List<Cancion> obtenerTodasLasCanciones() {
        return listaDeCanciones;
    }
```

Figura 2: Base de Datos

2.1. Metodos de Ordenamiento

Primero veremos los métodos de ordenamiento implementados dentro del código, y explicaremos uno y de qué forma funcionan. Este fue dividido en dos tipos de ordenamiento, el primero, fue ordenado por **Nombre**, es decir, en orden Lexográfico.

A continuacion, los veremos uno a uno:

```java
public void BubbleSortPorNombre() {
        int n = listaDeCanciones.size();
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (listaDeCanciones.get(j).nombre.compareTo(listaDeCanciones.get(j + 1).nombre) > 0) {
                    Cancion temp = listaDeCanciones.get(j);
                    listaDeCanciones.set(j, listaDeCanciones.get(j + 1));
                    listaDeCanciones.set(j + 1, temp);
                }
            }
        }
    }
```

Figura 3: BubbleSort por Nombre

El método **BubbleSortPorNombre** compara el nombre de cada canción con el nombre de su canción adyacente en la lista. Esto se realiza iterativamente para cada canción en la base de datos, asegurando que las canciones se ordenen en orden lexicográfico.

```java
public void InsertionSortPorNombre() {
        int n = listaDeCanciones.size();
        for (int i = 1; i < n; ++i) {
            Cancion key = listaDeCanciones.get(i);
            int j = i - 1;

            while (j >= 0 && listaDeCanciones.get(j).nombre.compareTo(key.nombre) > 0) {
                listaDeCanciones.set(j + 1, listaDeCanciones.get(j));
                j = j - 1;
            }
            listaDeCanciones.set(j + 1, key);
        }
    }
```

Figura 4: InsertionSort por Nombre

El siguiente método es **InsertionSortPorNombre**, el cual recorre la lista de canciones comenzando desde la segunda posición. Para cada canción, la compara con las canciones anteriores y la inserta en la posición correcta dentro de la sección ordenada de la lista. Esto asegura que las canciones se ordenen alfabéticamente por su nombre.

```java
public void MergeSortPorNombre() {
        listaDeCanciones = mergeSortNombre(listaDeCanciones);
    }

    private List<Cancion> mergeSortNombre(List<Cancion> lista) {
        if (lista.size() <= 1) {
            return lista;
        }

        int mid = lista.size() / 2;
        List<Cancion> left = new ArrayList<>(lista.subList(0, mid));
        List<Cancion> right = new ArrayList<>(lista.subList(mid, lista.size()));

        left = mergeSortNombre(left);
        right = mergeSortNombre(right);

        return mergeNombre(left, right);
    }

    private List<Cancion> mergeNombre(List<Cancion> left, List<Cancion> right) {
        List<Cancion> resultado = new ArrayList<>();
        int i = 0, j = 0;

        while (i < left.size() && j < right.size()) {
            if (left.get(i).nombre.compareTo(right.get(j).nombre) <= 0) {
                resultado.add(left.get(i));
                i++;
            }
            else {
                resultado.add(right.get(j));
                j++;
            }
        }

        while (i < left.size()) {
            resultado.add(left.get(i));
            i++;
        }

        while (j < right.size()) {
            resultado.add(right.get(j));
            j++;
        }

        return resultado;
    }
```

Figura 5: MergeSort por Nombre

El método **MergeSort**, podemos ver que se divide en 3 partes.

La primera es **MergeSortPorNombre** en el cual se inicializa Merge Sort para la lista de canciones por nombre. En esta se llama al método mergeSortNombre, el cual es recursivo y devuelve la lista ordenada por nombre.

Luego **mergeSortNombre** divide de forma recursiva la lista de canciones en mitades hasta que cada sublista contenga un elemento o esté vacía. Luego combina las sublistas ordenadas llamando a mergeNombre.

Por ultimo **mergeNombre** toma dos listas ordenadas (left y right) las cuales combina en una sola lista ordenada por nombre. Utilizando dos índices para comparar elementos y agregarlos a resultado en orden ascendente.

```java
public void QuickSortPorNombre() {
        quickSortNombre(listaDeCanciones, 0, listaDeCanciones.size() - 1);
    }

    private void quickSortNombre(List<Cancion> lista, int low, int high) {
        if (low < high) {
            int aux1 = partitionNombre(lista, low, high);
            quickSortNombre(lista, low, aux1 - 1);
            quickSortNombre(lista, aux1 + 1, high);
        }
    }

    private int partitionNombre(List<Cancion> lista, int low, int high) {
        Cancion aux2 = lista.get(high);
        int i = (low - 1);

        for (int j = low; j < high; j++) {
            if (lista.get(j).nombre.compareTo(aux2.nombre) <= 0) {
                i++;
                Cancion temp = lista.get(i);
                lista.set(i, lista.get(j));
                lista.set(j, temp);
            }
        }

        Cancion temp = lista.get(i + 1);
        lista.set(i + 1, lista.get(high));
        lista.set(high, temp);

        return i + 1;
    }
```

Figura 6: MergeSort por Nombre

Al igual que el método anterior, en este método **QuickSort**, se divide en 3 partes.

Primero, **QuickSortPorNombre**, inicia el proceso de ordenamiento Quick Sort para la lista de canciones por nombre. Esta llama al método quickSortNombre, que implementa la recursividad del algoritmo.

Luego **quickSortNombre** de forma recursiva ordena la lista de canciones por nombre.

Divide la lista en subgrupos en un auxiliar (aux1) obtenido del método partitionNombre, y luego ordena recursivamente cada subgrupo.

Finalmente, **partitionNombre** elige un auxiliar (aux2). Luego reorganiza la lista de modo que los elementos menores que el auxiliar estén a la izquierda y los mayores a la derecha. Retornando la posición final del auxiliar después de realizar el intercambio.

A continuación explicaremos los métodos de **Ordenamiento por Duración**, de forma más resumida, pero siguiendo la misma idea que los métodos ya creados anteriormente, con la única diferencia de que serán ordenados por el tiempo de cada canción dentro de la Base de Datos en milisegundos.

```java
public void BubbleSortPorDuracion() {
        int n = listaDeCanciones.size();
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (listaDeCanciones.get(j).duracion > listaDeCanciones.get(j + 1).duracion) {
                    Cancion temp = listaDeCanciones.get(j);
                    listaDeCanciones.set(j, listaDeCanciones.get(j + 1));
                    listaDeCanciones.set(j + 1, temp);
                }
            }
        }
    }
```

Figura 7: BubbleSort por Duracion

El método **BubbleSortPorDuracion** recorre repetidamente la lista, comparando pares de elementos adyacentes y realizando intercambios si están en el orden incorrecto. Este proceso va a continuar hasta que la lista esté completamente ordenada, según su tiempo de duración.

```java
public void InsertionSortPorDuracion() {
        int n = listaDeCanciones.size();
        for (int i = 1; i < n; ++i) {
            Cancion key = listaDeCanciones.get(i);
            int j = i - 1;

            while (j >= 0 && listaDeCanciones.get(j).duracion > key.duracion) {
                listaDeCanciones.set(j + 1, listaDeCanciones.get(j));
                j = j - 1;
            }
            listaDeCanciones.set(j + 1, key);
        }
    }
```

Figura 8: InsertionSort por Duracion

El método **InsertionSort** recorre la lista de canciones comenzando desde el segundo elemento. De forma que para cada canción, la va comparando con las canciones anteriores y la inserta en la posición correcta según su duración. Esto se irá repitiendo hasta que todas las canciones estén ordenadas de menor a mayor duración.

```java
public void MergeSortPorDuracion() {
        listaDeCanciones = mergeSortDuracion(listaDeCanciones);
    }

    private List<Cancion> mergeSortDuracion(List<Cancion> lista) {
        if (lista.size() <= 1) {
            return lista;
        }

        int mid = lista.size() / 2;
        List<Cancion> left = new ArrayList<>(lista.subList(0, mid));
        List<Cancion> right = new ArrayList<>(lista.subList(mid, lista.size()));

        left = mergeSortDuracion(left);
        right = mergeSortDuracion(right);

        return mergeDuracion(left, right);
    }

    private List<Cancion> mergeDuracion(List<Cancion> left, List<Cancion> right) {
        List<Cancion> resultado = new ArrayList<>();
        int i = 0, j = 0;

        while (i < left.size() && j < right.size()) {
            if (left.get(i).duracion <= right.get(j).duracion) {
                resultado.add(left.get(i));
                i++;
            } else {
                resultado.add(right.get(j));
                j++;
            }
        }

        while (i < left.size()) {
            resultado.add(left.get(i));
            i++;
        }

        while (j < right.size()) {
            resultado.add(right.get(j));
            j++;
        }

        return resultado;
    }
```

Figura 9: MergeSort por Duracion

El método **MergeSortPorDuracion** divide la lista de canciones en mitades recursivamente hasta que cada sublista contenga un solo elemento. Luego, combina estas sublistas ordenándolas por duración en un proceso donde se fusionan (*merge*). Este proceso se repite hasta que todas las sublistas estén combinadas en una lista ordenada por duración.

```java
public void QuickSortPorDuracion() {
        quickSortDuracion(0, listaDeCanciones.size() - 1);
    }

    private void quickSortDuracion(int low, int high) {
        if (low < high) {
            int aux1 = partitionDuracion(listaDeCanciones, low, high);
            quickSortDuracion(low, aux1 - 1);
            quickSortDuracion(aux1 + 1, high);
        }
    }

    private int partitionDuracion(List<Cancion> lista, int low, int high) {
        Cancion aux2 = lista.get(high);
        int i = (low - 1);

        for (int j = low; j < high; j++) {
            if (lista.get(j).duracion <= aux2.duracion) {
                i++;
                Cancion temp = lista.get(i);
                lista.set(i, lista.get(j));
                lista.set(j, temp);
            }
        }

        Cancion temp = lista.get(i + 1);
        lista.set(i + 1, lista.get(high));
        lista.set(high, temp);

        return i + 1;
    }
```

Figura 10: QuickSort por Duracion

El método **QuickSortPorDuracion** organiza la lista de canciones por duración, dividiéndola en particiones y ordenándolas de forma recursiva. Primero, elige un elemento auxiliar

de la lista y reorganiza los elementos para que todos los elementos menores al auxiliar estén a su izquierda y los mayores a su derecha. Se repetirá esto una y otra vez de manera recursiva hasta que toda la lista esté ordenada por duración.

2.2. Tiempos de ejecucion

A continuacion, veremos la forma en la cual fue implementado el calculo en milisegundos para cada uno de los timepos de ejecucion de los metodos de ordenamiento.

```java
public void TiempoBubbleSortPorNombre() {
        long start = System.nanoTime();
        BubbleSortPorNombre();
        long end = System.nanoTime();
        System.out.println("BubbleSort por nombre - Tiempo de ejecucion: " + (end - start) + " ns");
    }
```

```java
public void TiempoInsertionSortPorNombre() {
        long start = System.nanoTime();
        InsertionSortPorNombre();
        long end = System.nanoTime();
        System.out.println("InsertionSort por nombre - Tiempo de ejecucion: " + (end - start) + " ns");
    }
```

```java
public void TiempoMergeSortPorNombre() {
        long start = System.nanoTime();
        MergeSortPorNombre();
        long end = System.nanoTime();
        System.out.println("MergeSort por nombre - Tiempo de ejecucion: " + (end - start) + " ns");
    }
```

```java
public void TiempoQuickSortPorNombre() {
        long start = System.nanoTime();
        QuickSortPorNombre();
        long end = System.nanoTime();
        System.out.println("QuickSort por nombre - Tiempo de ejecucion: " + (end - start) + " ns");
    }
```

Figura 11: Calculo de tiempos de ejecucion

En los códigos mostrados anteriormente, podemos ver la forma en la cual es implementado el método *System.nanoTime();* el cual devuelve el tiempo actual en nanosegundos desde algún momento fijo de referencia, en este caso, se utiliza para medir el tiempo de referencia para cada Método de Ordenamiento por Nombre. Para el caso de los Ordenamientos por Duracion, se sigue la misma idea mostrada en los códigos, llamando al método de Ordenamiento en específico.

2.3. Arbol de Busqueda Binaria

Para hacer uso de este Árbol Binario se crea la clase correspondiente como se ve a continuación.

```java
class NodoArbol {
    Cancion cancion;
    NodoArbol left;
    NodoArbol right;

    public NodoArbol(Cancion cancion) {
        this.cancion = cancion;
        this.left = null;
        this.right = null;
    }
}
```

Figura 12: Clase NodoArbol

A continuación veremos los métodos creados a partir de esta clase.

```java
public void crearArbol() {
        rootPorNombre = null;
        rootPorDuracion = null;

        for (int i = 0; i < listaDeCanciones.size(); i++) {
            Cancion cancion = listaDeCanciones.get(i);

            rootPorNombre = insertarNodoPorNombre(rootPorNombre, cancion);
            rootPorDuracion = insertarNodoPorDuracion(rootPorDuracion, cancion);
        }
    }
```

Figura 13: Clase crearArbol

En el método **crearArbol**, se inicializan ambos árboles en null, de modo que están vacíos en un principio. Luego se recorre cada cancion dentro de la lista de canciones, para ser añadidas por los métodos *insertarNodoPorNombre* e *insertarNodoPorDuracion*, los cuales veremos a continuación.

```java
private NodoArbol insertarNodoPorNombre(NodoArbol nodo, Cancion cancion) {
        if (nodo == null) {
            return new NodoArbol(cancion);
        }

        if (cancion.nombre.compareToIgnoreCase(nodo.cancion.nombre) < 0) {
            nodo.left = insertarNodoPorNombre(nodo.left, cancion);
        } else {
            nodo.right = insertarNodoPorNombre(nodo.right, cancion);
        }
        return nodo;
    }
```

```java
private NodoArbol insertarNodoPorDuracion(NodoArbol nodo, Cancion cancion) {
        if (nodo == null) {
            return new NodoArbol(cancion);
        }

        if (cancion.duracion < nodo.cancion.duracion) {
            nodo.left = insertarNodoPorDuracion(nodo.left, cancion);
        } else {
            nodo.right = insertarNodoPorDuracion(nodo.right, cancion);
        }
        return nodo;
    }

```

Figura 14: Metodos de Insecion por Nombre y Duracion

De las imágenes anteriores, podemos ver como, dependiendo del caso, son agregadas a los nodos izquierdos o derechos de la raíz dependiendo del método implementado. Para el caso de **Insertar Nombre por Duración**, inserta la canción en el árbol de manera recursiva, moviéndose hacia abajo por el árbol, dependiendo de si el nombre de la canción a insertar es menor o mayor que el nombre de la canción en el nodo actual. Para el caso de la **Insercion por Duración**, es la misma idea, se inserta la canción en el árbol de manera recursiva, de forma que se mueve por el árbol, y en caso de que la duración de la canción a insertar es menor o mayor que la duración de la canción en el nodo actual.

```java
public int buscarCancion(String nombre) {
        if (nombre == null || nombre.isEmpty()) {
            return -1;
        }
        return buscarEnArbolPorNombre(rootPorNombre, nombre);
    }

    private int buscarEnArbolPorNombre(NodoArbol nodo, String nombre) {
        if (nodo == null) {
            return -1;
        }

        String nombreNormalizado = nombre;
        String nombreNodoNormalizado = nodo.cancion.nombre;

        if (nombreNormalizado.equals(nombreNodoNormalizado)) {
            return nodo.cancion.id;
        }
        else if (nombreNormalizado.compareTo(nombreNodoNormalizado) < 0) {
            return buscarEnArbolPorNombre(nodo.left, nombre);
        }
        else {
            return buscarEnArbolPorNombre(nodo.right, nombre);
        }
    }
```

Figura 15: Buscar Cancion

En el método anterior **buscarCancion** (Por Nombre), se realiza una búsqueda binaria dentro del árbol basándose en la comparación de nombres de canciones. Primero verificamos que el nombre proporcionado no esté vacío y luego llama al método *buscarEnArbolPorNombre*, pasando la raíz del árbol (rootPorNombre) y el nombre de la canción a buscar. Luego el método realiza una búsqueda recursiva en el árbol, en el cual, si encuentra la canción con el nombre exacto, devuelve su identificador (id); de lo contrario, decide continuar la búsqueda en el subárbol izquierdo o derecho, dependiendo del caso.

```java
public List<Integer> buscarCancionesPorDuracion(double duracion) {
        List<Integer> ids = new ArrayList<>();
        buscarEnArbolPorDuracion(rootPorDuracion, duracion, ids);
        return ids;
    }

    private void buscarEnArbolPorDuracion(NodoArbol nodo, double duracion, List<Integer> ids) {
        if (nodo == null) {
            return;
        }

        if (nodo.cancion.duracion == duracion) {
            ids.add(nodo.cancion.id);
        }

        buscarEnArbolPorDuracion(nodo.left, duracion, ids);
        buscarEnArbolPorDuracion(nodo.right, duracion, ids);
    }
```

Figura 16: Buscar Cancion Por Duracion

Para este método, se recibe una duración como parámetro y se devuelve una lista de ids de canciones que tienen la duración dada. Luego se inicializa una lista vacía donde se almacenarán las ids encontradas. Luego de esto se llama al metodo **buscarEnArbolPorDuracion** el cual realiza una búsqueda recursiva dentro del árbol. En el cual, si la duración de la canción en el nodo actual coincide con la duración buscada, se añade el id de esa canción a la lista ids. Por último, la función se llama recursivamente para explorar los subárboles izquierdo y derecho del nodo actual.

3. Análisis

Uno de los objetivos de este proyecto es desarrollar un buscador de música eficiente que permita buscar canciones por nombre y duración. Para lograr esto, hemos implementado varios métodos de búsqueda y ordenamiento para analizar su comportamiento. A continuación, presentaremos una evaluación del rendimiento de estos métodos tras la implementación de nuestro archivo *CSV*.

Al ejecutar nuestro programa, se obtienen los siguientes resultados:

```
=== TIEMPOS DE EJECUCION ===

- Por Nombre:
Quick Sort por nombre - Tiempo de ejecución: 39826100 ns
Bubble Sort por nombre - Tiempo de ejecución: 22500615500 ns
Insertion Sort por nombre - Tiempo de ejecución: 6486700 ns
Merge Sort por nombre - Tiempo de ejecución: 29923000 ns

- Por Duracion:
Quick Sort por duración - Tiempo de ejecución: 13749300 ns
Bubble Sort por duración - Tiempo de ejecución: 1038198800 ns
Insertion Sort por duración - Tiempo de ejecución: 6275600 ns
Merge Sort por duración - Tiempo de ejecución: 15611200 ns
```

Figura 17: Timepo de Ejecución en código.

Para poder apreciar bien la diferencia en cuanto a rendimiento de cada uno de estos metodos, los llevaremos a una tabla.

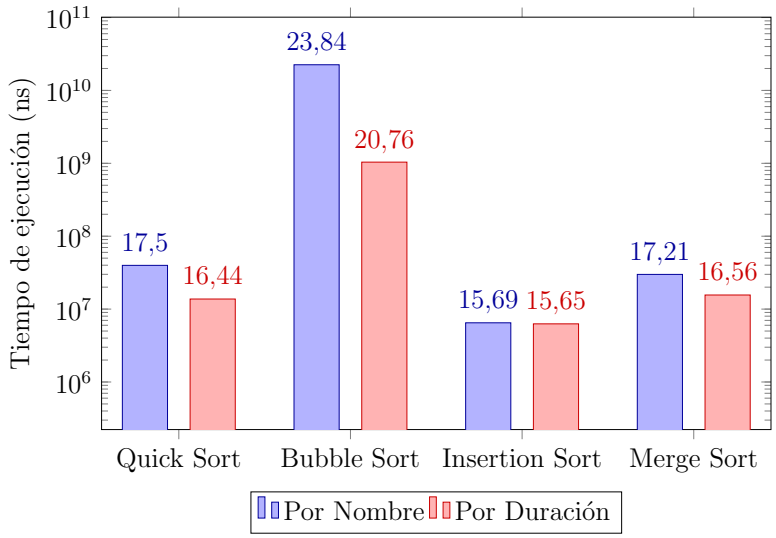


Figura 18: Tiempos de ejecución de algoritmos de ordenamiento.

4. Conclusión

En resumen, el proyecto logró el objetivo principal de desarrollar un buscador de música. Durante el desarrollo, se apreciaron varios aspectos, incluyendo el análisis del tiempo de ejecución de los diferentes métodos implementados, además de la implementación de un Árbol de Búsqueda Binaria (BST).
Comprendí de mejor manera la implementación de métodos de ordenamiento como Bubble Sort, Insertion Sort, Merge Sort y Quick Sort, los cuales no solo fueron aplicados para ordenar la lista de canciones, sino que también se utilizó su rendimiento para evaluar su eficiencia en distintos casos. Además, gracias al BST entendí cómo estructuras de datos más complejas pueden optimizar las operaciones de búsqueda en este caso, ya que facilitó la búsqueda por nombre y duración de las canciones.
Por ultimo, agregar que aprendí la forma de trabajar con un archivo CSV dentro del código, y de que manera se implementa.